


VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

**School of Computer Science and Engineering (SCOPE)**

**BCSE306L – Artificial Intelligence**

Slot: C1 + TC1 · Project Document (Digital Assignment)

Project Title

# CineMind AI: A Local-First, Explainable AI Assistant for Video Pre-Editing

Submitted by


| Name | Registration No. |
|---|---|
| Rishith Raj G | 24BCE0175 |
| Hrishikesh Acharya | 24BDS0231 |
| Krish Singh | 24BDS0235 |


October 2026

# Contents

1. Structured Abstract....3
2. Introduction....3
3. Background / Related Work....4
4. Summary of Limitations / Gaps....8
5. Primary Contributions of the Project....9
6. Individual Contributions....9
7. Tools and Technologies Used....10
8. Methodology Used....11
9. Dataset Used....12
10. Proposed Approach....13
11. Result Analysis and Discussion....14
12. Comparative Analysis....18
13. User Manual....19
14. References....25
15. Appendix....26

*(Right-click the table and choose Update Field in Word if page numbers are missing.)*

# 1. Structured Abstract

**Background:** Editors spend a large share of post-production organising raw footage: finding usable shots, reading through takes and building a first assembly. Commercial AI features that help are mostly cloud-based, paid, or locked to one editing program.

**Objective:** To build a local-first assistant that turns an unorganised folder of clips into a searchable, ranked library and an editable AI-assisted rough cut that opens in DaVinci Resolve, while explaining every decision and leaving creative control with the editor.

**Methods:** A modular pipeline combines PySceneDetect shot detection, technical quality measures, faster-whisper transcription, YuNet/SFace face grouping, CLIP/SigLIP picture and MiniLM text embeddings, zero-shot tags, hybrid search with Reciprocal Rank Fusion and an explainable weighted ranking, take detection, script-to-footage alignment and a rule-based rough-cut engine exporting FCP7 XML, EDL and OpenTimelineIO. Everything runs on a CPU laptop (Apple M1, 8 GB).

**Results:** On public benchmarks run through our own pipeline: 4.70 % word error rate (Whisper small, Mini LibriSpeech), face-grouping B-cubed F1 0.993 (LFW held-out), tag precision 0.90–1.00 for eight enabled tags, quality-flag F1 0.954–0.997. On the users' footage, hybrid search reached nDCG@10 0.907 and MRR 0.971 (17 queries), script alignment found 83 % of transcribed lines (Tears of Steel), and rough cuts were built in 0.2 s within 10 % of the requested length. FCP7 XML timelines imported correctly into DaVinci Resolve 21.1.

**Conclusion:** Explainable, local AI can take over the organising work of pre-editing on ordinary hardware. Judgements on the team's own footage are still few and were made with an AI assistant (Claude) rather than by independent viewers; they are reported as such.

**Keywords:** video pre-editing, multimodal retrieval, speech recognition, face clustering, explainable ranking, rough cut, local-first AI

# 2. Introduction

## 2.1 Background and Problem Definition

A short film or vlog shoot easily produces hundreds of clips: several takes of each line, phone clips at mixed frame rates, out-of-focus or badly exposed shots, and hours of speech in more than one language. Before any creative editing starts, an editor has to watch everything, log it, discard unusable material, find each line and its best take, and assemble a first cut. Existing AI help is fragmented: transcription-based editing in Adobe Premiere Pro, script-based assembly in DaVinci Resolve 20 Studio, or third-party search plug-ins, each tied to one editor, often paid or cloud-based, and rarely able to say why a shot was chosen. Research systems address single tasks (shot detection, scene segmentation, cut plausibility, moment retrieval) on movie datasets rather than on a filmmaker's raw, unedited footage. The problem we address is therefore: how can raw footage be organised, searched, ranked and turned into a first assembly automatically, on the filmmaker's own computer, in a way the editor can inspect and override?

## 2.2 Objective and Methodology

Our objective is CineMind AI, a local-first pre-editing assistant that never modifies source footage and is explicitly not an auto-editor: every generated edit is labelled 'AI-assisted rough cut. Human review required.' The system is a modular monolith: a FastAPI service bound to 127.0.0.1, a single background worker, SQLite (WAL mode, FTS5 full-text search) and a React user interface. Each clip passes through cached, resumable analysis stages keyed by its content hash, parameters and model version: metadata and proxy generation with FFmpeg, shot detection with PySceneDetect, keyframes, technical quality (focus, exposure, contrast, loudness, clipping), transcription with faster-whisper including word timings and voice-activity detection, face detection and recognition with YuNet and SFace followed by constrained agglomerative clustering into actors, picture embeddings (OpenCLIP ViT-B/32 on our lite tier, SigLIP on larger tiers), sentence embeddings (MiniLM) and zero-shot tags whose thresholds were calibrated on labelled public data.

On top of these stages we built the project's own contributions: hybrid search fusing four result lists (BM25 keywords, labels, meaning of speech, picture) with Reciprocal Rank Fusion; an explainable ranking with a per-signal score breakdown, what-if weight sliders and a duplicate penalty (maximal marginal relevance); take detection with a transparent best-take suggestion; script-to-footage alignment; and a rough-cut engine that reads a one-sentence brief, selects, trims and orders shots and exports FCP7 XML, CMX3600 EDL and OpenTimelineIO with markers and subtitles. Every threshold is either evaluated by a committed script in eval/ or explicitly labelled as a convention.

### 2.3 Results, Outcomes and Analysis

Evaluated through the application's own code, transcription reached a 4.70 % word error rate on a public English sample, actor grouping a B-cubed F1 of 0.993 on Labeled Faces in the Wild, and the zero-shot tags 0.90–1.00 held-out precision for the eight tags left switched on (six were switched off for falling below 0.85). On the team's own footage, the picture list alone and the fused hybrid search both ranked relevant shots first (nDCG@10 0.925 and 0.907), showing that for visual queries the picture embedding dominates while fusion keeps spoken-word queries possible. Script alignment found 19–20 of 24 lines that Whisper had transcribed, so the speech model, not the matcher, is the main limit. Rough cuts were built in about 0.2 s (28.7 s when models first load) and stayed within 10 % of the requested length, and the exported XML opened correctly in DaVinci Resolve 21.1. Search relevance and take labels on our footage were judged with an AI assistant (Claude) from keyframes and transcripts rather than by independent viewers; this is stated, and those figures are treated as indicative.

## 3. Background / Related Work

Each team member studied five research articles (2020 onwards) and two or three tools or reports. The six paragraphs below discuss them by theme with their flaws; Table 1 summarises the research articles in the prescribed template and Table 2 the tools.

Speech recognition is the backbone of dialogue-aware editing. Radford et al. [1] trained Whisper on 680,000 hours of weakly supervised multilingual audio and showed robust zero-shot transcription close to fully supervised systems; however, its timestamps are segment-level and it can produce fluent text on noise. Bain et al. [2] addressed exactly this with WhisperX, adding voice-activity detection and

forced phoneme alignment for word-level timestamps and a twelve-fold batched speed-up, but it depends on an extra alignment model per language and was evaluated on clean long-form benchmarks rather than noisy field recordings. Our pipeline uses faster-whisper with built-in VAD and word timings and adds uncertainty marking so that hallucinated lines are hidden.

Joint vision–language models make free-text search of footage possible. Radford et al. [3] introduced CLIP, trained contrastively on 400 million image–text pairs, enabling zero-shot classification and retrieval; Zhai et al. [4] replaced the softmax contrastive loss with a pairwise sigmoid loss (SigLIP), which scales better and works with smaller batches. Luo et al. [5] transferred CLIP to video–text retrieval (CLIP4Clip) and found mean-pooled frame features already strong. All three are trained on web images or edited video and do not report calibrated, per-tag precision for raw footage; CLIP-style scores are also uncalibrated across prompts. We therefore calibrate every tag threshold on labelled public photos and switch off tags that fall below 0.85 precision.

Face analysis and shot boundaries are classic building blocks. Deng et al. [6] proposed RetinaFace, a single-stage detector with landmark supervision achieving state-of-the-art accuracy on WIDER FACE, but at a computational cost unsuited to an 8 GB laptop. Wu et al. [7] designed YuNet, a millisecond-level detector with under 100k parameters, trading some accuracy on hard faces for speed. Souček and Lokoč [8] presented TransNet V2, a 3D-CNN shot-transition detector trained on synthetic transitions; it targets edited broadcast video, while raw phone footage rarely contains cuts and produces false boundaries on fast motion. We use YuNet + SFace with quality filtering and constrained clustering, and PySceneDetect's adaptive detector, and we report false cuts on our own footage.

Movie-understanding datasets supply structure beyond shots. Huang et al. [9] released MovieNet, 1,100 movies with 1.1 M character boxes, 42 K scene boundaries and cinematic-style tags; Rao et al. [10] proposed a local-to-global multimodal scene segmentation model trained on MovieScenes (21 K scenes from 150 movies). Both rely on professionally edited films, copyright-restricted frames and heavy GPU training, so they do not transfer directly to an individual filmmaker's raw clips; our scene grouping is a transparent similarity rule that the editor can split, join and rename.

Learning-based editing research targets individual editing decisions. Argaw et al. [11] built the Anatomy of Video Editing benchmark (5,591 scenes, 196,176 shots, 1.5 M labels) for AI-assisted editing tasks; Pardo et al. [12] learned to judge cut plausibility from about 255 K cuts in 10 K edited videos; Shen et al. [13] recommended transition effects with a multimodal transformer (AutoTransition). Lei et al. [14] introduced QVHighlights, over 10 K videos with natural-language moment and highlight annotations, and the Moment-DETR baseline. These works model one step of editing on edited, mostly English web or movie content, need large GPUs, and do not output an editable NLE timeline. Wang et al. [15] distilled compact sentence encoders (MiniLM) that make on-device text embeddings practical, which we use for meaning-of-speech search.

Commercial tools and open libraries show demand but leave gaps. Adobe's text-based editing in Premiere Pro [16] transcribes media so editors cut by editing text, but runs inside one paid editor without shot ranking or explanations. Blackmagic's DaVinci Resolve 20 adds IntelliScript [17], which builds a timeline from a script by matching transcribed audio, but it is a Studio (paid) feature without

visible scores. Jumper [18] offers local natural-language and spoken-word search inside editors but does not build cuts or judge quality. Open-source libraries such as PySceneDetect [19] and OpenTimelineIO [20] provide reliable components without the AI layer. CineMind combines these capabilities in one free, local, explainable tool that exports to any NLE through FCP7 XML, EDL or OTIO.

*Table 1. Literature survey of research articles (2020 onwards)*

| Sl# | Articles | Work Done | Dataset | Gaps / Limitations |
|---|---|---|---|---|
| 1 | Radford et al. [1]<br>ICML (PMLR 202), 2023 | • Problem: speech recognition that generalises without fine-tuning<br>• Method: encoder–decoder Transformer trained on 680K h weakly supervised multilingual audio<br>• Result: competitive with supervised systems zero-shot, robust to accents and noise | Web-collected audio–transcript pairs (680,000 h; 96 languages + translation); not released | • Training data not public; hallucinates text on non-speech audio<br>• Segment-level timestamps; large models need a GPU |
| 2 | Bain et al. [2]<br>Interspeech, ISCA, 2023 | • Problem: inaccurate and missing word timestamps in Whisper<br>• Method: VAD cut & merge + forced phoneme alignment (wav2vec2)<br>• Result: state-of-the-art word segmentation, 12× faster batched inference | TED-LIUM, Kincaid46, AMI, Switchboard (long-form English audio) | • Needs a separate alignment model per language<br>• Evaluated mostly on clean English speech, not field recordings |
| 3 | Radford et al. [3]<br>ICML (PMLR 139), 2021 | • Problem: visual models limited to fixed label sets<br>• Method: contrastive image–text pre-training (CLIP)<br>• Result: strong zero-shot transfer, e.g. ImageNet without labels | WIT: 400 M image–text pairs from the web (not released) | • Scores uncalibrated across prompts; social biases from web data<br>• Single images only, no temporal modelling |
| 4 | Zhai et al. [4]<br>ICCV, IEEE/CVF, 2023 | • Problem: softmax contrastive loss needs huge global batches<br>• Method: pairwise sigmoid loss (SigLIP)<br>• Result: better zero-shot accuracy at small batch sizes, cheaper training | WebLI (multilingual web image–text pairs; not public) | • Non-public training data; heavy TPU training<br>• Zero-shot scores still need per-task thresholds |
| 5 | Luo et al. [5]<br>Neurocomputing, Elsevier, 2022 | • Problem: transfer image–text knowledge to video retrieval<br>• Method: CLIP frame encoder with mean-pooling / temporal Transformer (CLIP4Clip)<br>• Result: state-of-the-art text-to-video retrieval on MSR-VTT, MSVD, LSMDC | MSR-VTT (10K clips, 200K captions), MSVD, LSMDC, ActivityNet, DiDeMo | • Benchmarks of short edited web clips, English captions<br>• Fine-tuning needs GPUs; no quality or explanation signals |
| 6 | Deng et al. [6]<br>CVPR, IEEE/CVF, 2020 | • Problem: accurate face localisation in the wild<br>• Method: single-stage detector with extra landmark and dense 3D supervision (RetinaFace) | WIDER FACE (32,203 images, 393,703 faces) + 5-point landmark labels | • Heavy backbone; slow on CPU<br>• Detection only, no identity grouping across clips |

| Sl# | Articles | Work Done | Dataset | Gaps / Limitations |
|---|---|---|---|---|
| | | • Result: state-of-the-art AP on WIDER FACE hard | | |
| 7 | Wu et al. [7]<br>Machine Intelligence Research, Springer, 2023 | • Problem: face detection on edge devices<br>• Method: tiny anchor-free network (YuNet), < 100K parameters<br>• Result: millisecond inference with competitive WIDER FACE accuracy | WIDER FACE (easy / medium / hard splits) | • Lower recall on hard, small or occluded faces<br>• Reported accuracy differs between paper and model card |
| 8 | Souček and Lokoč [8]<br>arXiv preprint, 2020 | • Problem: universal shot-transition detection<br>• Method: dilated 3D CNN trained on synthetic transitions (TransNet V2)<br>• Result: state-of-the-art F1 on ClipShots, BBC Planet Earth, RAI | ClipShots (4,039 videos), IACC.3 / synthetic transitions; BBC, RAI test sets | • Trained for edited video; raw handheld footage triggers false cuts<br>• Needs TensorFlow / GPU for speed |
| 9 | Huang et al. [9]<br>ECCV, Springer, 2020 | • Problem: no holistic dataset for movie understanding<br>• Method: MovieNet with multi-level annotations and benchmarks<br>• Result: baselines for scene segmentation, character identification, cinematic style | MovieNet: 1,100 movies, 1.1 M character boxes, 42 K scene boundaries, 92 K style tags | • Copyrighted films; frames distributed under restrictions<br>• Professionally edited cinema, unlike raw footage |
| 10 | Rao et al. [10]<br>CVPR, IEEE/CVF, 2020 | • Problem: segmenting films into semantic scenes<br>• Method: local-to-global multimodal model (place, cast, action, audio)<br>• Result: clearly better scene-boundary AP than earlier methods | MovieScenes: 21 K scene segments from 150 movies | • Needs pre-computed multimodal features and GPU training<br>• Only edited films; no user correction loop |
| 11 | Argaw et al. [11]<br>ECCV, Springer, 2022 | • Problem: AI-assisted editing lacks benchmarks beyond effects<br>• Method: Anatomy of Video Editing dataset and task suite<br>• Result: baselines for shot attributes, assembly and next-shot selection | AVE: 5,591 movie scenes, 196,176 shots, > 1.5 M labels (8 properties) | • Movie scenes only, no raw takes<br>• Baselines far from editor quality; no end-to-end tool |
| 12 | Pardo et al. [12]<br>ICCV, IEEE/CVF, 2021 | • Problem: deciding where cuts are plausible<br>• Method: contrastive audio-visual model trained on real vs. artificial cuts<br>• Result: outperforms baselines; preferred in human studies | About 10 K edited videos with ~255 K extracted cuts | • Learns from edited web video; style bias<br>• Scores cut pairs only; no shot selection or export |
| 13 | Shen et al. [13]<br>ECCV, Springer, 2022 | • Problem: choosing transitions between shots | Collected edited videos with labelled transitions (released on GitHub) | • Transitions only, not cut selection |

| Sl# | Articles | Work Done | Dataset | Gaps / Limitations |
|---|---|---|---|---|
| | | • Method: multimodal transformer retrieving transition embeddings (AutoTransition)<br>• Result: comparable to professional editors in a user study | | • Trained on short social-media style edits |
| 14 | Lei et al. [14] NeurIPS, 2021 | • Problem: retrieving moments and highlights from text queries<br>• Method: QVHighlights dataset + Moment-DETR transformer<br>• Result: competitive baseline; ASR pre-training helps | QVHighlights: > 10 K YouTube videos, free-form queries, 5-point saliency | • English YouTube vlogs/news only<br>• Needs GPU training; no notion of technical quality or takes |
| 15 | Wang et al. [15] NeurIPS, 2020 | • Problem: large Transformers too slow for deployment<br>• Method: deep self-attention distillation (MiniLM)<br>• Result: keeps most of the teacher's accuracy at a fraction of the size | English Wikipedia + BookCorpus (pre-training); GLUE, SQuAD 2.0 (evaluation) | • English-centric small model<br>• Sentence vectors weaker on non-English transcripts |

*Table 2. Tools, blogs and reports studied*

| Tool / report | What it does | Gaps / Limitations |
|---|---|---|
| Adobe Premiere Pro Text-Based Editing [16], Adobe blog, 2023 | Transcribes source media; editors cut, copy and rearrange text to edit the timeline; captions from transcript. | Inside one paid editor; cloud account; no shot ranking, quality check or explanation. |
| DaVinci Resolve 20 IntelliScript [17], Blackmagic Design, 2025 | Builds a timeline from a script by matching transcribed audio; alternate takes on extra tracks. | Studio (paid) feature; no visible match scores; no search or quality ranking. |
| Jumper [18], Witchcraft Software, 2024 | Local natural-language, spoken-word and 'find similar' search inside Premiere / Final Cut. | Search only: no rough cut, takes or quality; paid subscription; closed source. |
| PySceneDetect [19] and OpenTimelineIO [20], open source | Reliable shot detection and an interchange timeline model with adapters (FCP XML, EDL). | Building blocks only; no AI understanding, ranking or user interface. |

## 4. Summary of Limitations / Gaps

- **G1.** Raw footage is under-served: most methods and datasets [8]–[14] use professionally edited films or web videos, not the unedited, mixed-frame-rate, multi-take clips a filmmaker actually has.
- **G2.** Single-task scope: each work solves one step (transcription, shot boundaries, scenes, cut plausibility, transitions, moment retrieval); none links organising, searching, ranking and assembling into an editable NLE timeline.
- **G3.** Opaque decisions: CLIP-style and learned scores are uncalibrated and unexplained [3]–[5]; commercial tools [16]–[18] show results without the reasons behind them.
- **G4.** Hardware and access barriers: strong models need GPUs or cloud services [1], [4], [6], [8]; commercial features are paid or tied to one editor [16]–[18].

- **G5.** Weak handling of real-world audio: ASR hallucinates text on noise and silence [1], [2], and non-English speech is poorly transcribed by small models.
- **G6.** Little evaluation on the user's own material and no correction loop: published figures are on benchmarks, and few systems let the editor correct a grouping and keep that correction.

## 5. Primary Contributions of the Project

- **End-to-end local pipeline for raw footage (gaps 1, 2, 4):** From an untouched footage folder to a DaVinci Resolve timeline on an 8 GB CPU laptop: ingest, shots, quality, transcripts, actors, tags, scenes, search, takes, script and rough cut in one modular application. No cloud and no paid API.
- **Explainable hybrid search and ranking (gap 3):** Four retrieval lists fused with Reciprocal Rank Fusion, then a weighted ranking whose per-signal breakdown (relevance, quality, selects, person, scene, duplicate penalty) is shown on every result, with what-if weight sliders.
- **Calibrated, honest AI outputs (gaps 3, 5, 6):** Tag thresholds set at 0.9 precision on labelled public data, and tags below 0.85 switched off. Quality flags are calibrated on synthetic degradations. Uncertain transcript lines are hidden. Every number in the documentation comes from a committed evaluation script.
- **Take detection and best-take suggestion (gaps 2, 3):** Repeated takes are grouped by transcript words, picture similarity and adjacency. The best take is suggested from focus, exposure, audio, completeness and flubs, with each signal shown and the editor able to overrule it.
- **Script-to-footage alignment (gaps 1, 2):** A screenplay (Fountain or plain text) is matched line by line against the transcripts. It finds up to five takes per line, rejects lines that were never filmed, and builds a script-ordered assembly.
- **AI-assisted rough cut with NLE export (gaps 2, 4):** A one-sentence brief becomes a trimmed, ordered timeline with one take per group, whole sentences and the requested length. It exports to FCP7 XML, EDL, OTIO and SRT with a reason marker on every clip, and was confirmed to import into DaVinci Resolve 21.1.
- **Human-in-the-loop corrections that persist (gap 6):** Scene splits and joins, actor merges and splits, take corrections, selects/rejects and search judgements are stored by stable ids. They survive re-processing and are logged as feedback for evaluation and future learning.

## 6. Individual Contributions

All three members contributed equally in effort. The work was divided by area as below; design reviews, testing on our footage and the final report were done together.

### Rishith Raj G (24BCE0175)

- System architecture: FastAPI service, SQLite schema with Alembic migrations, job queue and worker with cached, resumable stages, security rules (localhost binding, path validation).
- Ingest pipeline: ffprobe metadata, content hashing, proxies, thumbnails, waveforms; shot detection and keyframes; timeline export to FCP7 XML, CMX3600 EDL and OpenTimelineIO, and the manual DaVinci Resolve import tests.
- Speech: faster-whisper transcription with word timings, uncertainty marking, per-project languages; script-to-footage alignment and the script assembly.
- Documentation: introduction, methodology, proposed approach, tools and technologies; literature study of [1], [2], [5], [14], [15], [16], [17].

### Hrishikesh Acharya (24BDS0231)

- Computer vision: technical quality metrics (focus, exposure, contrast) and audio QC, with threshold calibration on synthetically degraded frames.
- Faces and actors: YuNet/SFace detection, tracking, constrained agglomerative clustering and the 'same person?' review loop; evaluation on LFW (purity, ARI, NMI, B-cubed).
- Embeddings and tags: OpenCLIP/SigLIP picture vectors, MiniLM text vectors, zero-shot tag calibration on Places365 and CODaN, scene grouping, and take detection with best-take signals.
- Documentation: dataset description, result analysis and figures, summary of gaps; literature study of [3], [4], [6], [7], [8], [18], [19].

### Krish Singh (24BDS0235)

- Frontend: React + TypeScript + Tailwind interface (library, clip player, shots/scenes, actors, transcripts, search, takes, script, timeline editor with drag-and-drop, export), generated API client, Vitest and Playwright tests.
- Search and ranking: four retrieval lists, Reciprocal Rank Fusion, weighted explainable ranking with MMR duplicate penalty and what-if sliders, search evaluation (P@K, nDCG@10, MRR, ablations).
- Rough-cut engine: brief parsing, candidate selection, speech-aware trimming, length fitting, ordering, swap suggestions and SRT/marker export; timing evaluation.
- Documentation: comparative analysis, user manual with screenshots, presentation and demonstration video; literature study of [9], [10], [11], [12], [13], [20].

## 7. Tools and Technologies Used

### 7.1 Hardware Requirements

| Item | Specification |
|---|---|
| Development and test machine | Apple MacBook with M1 (8 cores), 8 GB RAM, Apple GPU (MPS); external SSD (about 480 GB free). |
| Minimum to run | Any 64-bit computer with 8 GB RAM and a few GB of disk; CPU is enough (model tier 'lite'). An NVIDIA GPU is optional and selects larger models. |
| Operating systems | macOS (tested), Linux and Windows (cross-platform Python and Node tooling). |

*Table 3. Hardware*

### 7.2 Software Requirements

| Layer | Software |
|---|---|
| Language / runtime | Python 3.11+ (3.13 used), uv package manager; Node.js 20+ with pnpm |
| Backend | FastAPI, Pydantic v2, pydantic-settings, SQLAlchemy 2.0, Alembic, SQLite (WAL, FTS5) |
| Video / audio | FFmpeg 9 / ffprobe (subprocess, argument lists), OpenCV, PySceneDetect |
| AI models | faster-whisper 1.2 (CTranslate2) with Whisper small; OpenCV YuNet + SFace; OpenCLIP ViT-B/32 (LAION-2B), SigLIP base; MiniLM-L6 sentence encoder; PyTorch 2 + Hugging Face transformers; scikit-learn; FAISS / numpy vector index |
| Timeline / export | OpenTimelineIO (internal model), own FCP7 XML, CMX3600 EDL, SRT writers |
| Frontend | React, Vite, TypeScript (strict), Tailwind CSS, TanStack Query, React Router, Zustand, dnd-kit, wavesurfer.js |
| Quality | pytest (356 tests), Vitest + Testing Library (95 tests), Playwright end-to-end, ruff, pyright, ESLint |
| Target editor | DaVinci Resolve 21.1 (free edition) for import tests |

*Table 4. Software stack*

# 8. Methodology Used

## Shot detection (PySceneDetect) [19]

The adaptive content detector compares HSV frame differences with a rolling average; a cut is placed where the difference spikes. Shots shorter than 0.5 s are merged. Up to three sharpest keyframes per shot are saved.

## Technical quality

Focus is a re-blur ratio of Laplacian energy with an absolute floor, exposure uses mean luma and clipped/crushed pixel shares, contrast uses luma spread, and audio uses EBU R128 loudness (LUFS), clipping and silence. Thresholds were calibrated on real frames with synthetic blur and exposure changes.

## Speech recognition (Whisper / faster-whisper) [1]

A Transformer encoder–decoder transcribes 16 kHz audio after voice-activity detection, giving word-level timestamps. Segments with low average log-probability or a high compression ratio are marked uncertain, a common symptom of hallucination [2].

## Faces and actors (YuNet + SFace) [7]

Faces are detected at 2 frames/s, filtered by size, turn and sharpness, embedded into 128-d vectors and tracked within a shot. Tracks are grouped by average-linkage agglomerative clustering on cosine distance, with cannot-link constraints for faces in the same frame and for 'not the same person' answers.

## Picture and text embeddings (CLIP / SigLIP, MiniLM) [3], [4], [15]

Keyframes are embedded by a vision–language model and transcript lines by a sentence encoder; vectors are L2-normalised and searched by inner product (FAISS or numpy).

## Zero-shot tags

Each tag is a group of text prompts. Softmax over the group gives a score, and a tag is shown only above a threshold that reached 0.9 precision on a labelled public set (Places365, CODaN).

## Hybrid search and Reciprocal Rank Fusion [21]

The BM25 keyword list (SQLite FTS5), label list, meaning-of-speech list and picture list each rank shots. RRF sums 1/(k + rank) with k = 60. The final ranking is a weighted sum of relevance, quality, selection, person and scene, minus a duplicate penalty chosen greedily (MMR, cosine > 0.85). Every term is returned for display.

## Take detection and best take

Shot pairs are linked when their transcript words match (difflib ratio ≥ 0.6, pictures ≥ 0.5), or, without speech, when their pictures match (≥ 0.9) and they are close in order. Average linkage at a mean pair

score of 0.75 prevents chaining. The best take is the weighted mean of focus, exposure, audio, completeness and cleanliness.

### Script alignment

Dialogue lines are matched against windows of 1–3 transcript segments by word alignment (Ratcliff/Obershelp) and by MiniLM cosine. The default is meaning only with a 0.3 limit, chosen on a development half.

### Rough-cut engine and export

The brief is parsed by rules (length, order). Candidates come from search and ranking, then pass a quality ≥ 0.5 and length ≥ 1.5 s filter and the one-take-per-group rule. Each clip is trimmed to whole transcript lines plus 0.25 s, or 6 s around the match, filled to the target length ±10 %, ordered, and stored as an OpenTimelineIO-style sequence exported to FCP7 XML / EDL / OTIO with rational frame rates [20].

## 9. Dataset Used

CineMind analyses the user's own footage, which needs no training data; datasets are used to evaluate and calibrate each AI component. All public datasets were downloaded once by the scripts in eval/ into a git-ignored folder and are used through the application's own code.

| Dataset (source) | Used for | Properties |
|---|---|---|
| Team footage (own recording) | End-to-end runs, search, takes, rough cut | 25 clips, 532 s, 2.23 GB; phone and stock video; H.264 and HEVC; 1080p–4K, portrait and landscape; 24–120 fps (2 variable frame rate); English, Kannada and Hindi speech; 76 shots, 158 keyframes, 31 transcript segments, 475 faces, 15 actors, 72 scenes |
| Mini LibriSpeech dev-clean-2 (openslr.org/31, CC BY 4.0) | Word error rate of transcription | Read English audiobook speech, 16 kHz FLAC with reference transcripts; fixed random sample of 1,682 words |
| Labeled Faces in the Wild (vis-www.cs.umass.edu/lfw) | Actor grouping purity, ARI, NMI, B-cubed | 13,233 face photos of 5,749 people; we used people with ≥ 2 photos, split into a tuning half (2,295 photos / 412 people) and held-out half (2,307 / 411) |
| Places365 validation (places2.csail.mit.edu) | Place and indoor/outdoor tag thresholds | 36,500 scene photos, 365 categories (100 each); mapped to our tags |
| CODaN (Common Objects Day and Night) | Day / night tag thresholds | Object crops captured in daylight and in low light; 1,250 + 1,250 held-out images |
| Tears of Steel (Blender Foundation, CC BY 3.0) | Script alignment accuracy | 12-minute live-action short film (734 s) with English subtitles used as the script; split into development and test halves (48 test lines) |

*Table 5. Datasets*

Sample of the team footage (metadata read by ffprobe; every column is stored per clip):

| File | Duration (s) | Resolution | Video / audio codec | Frame rate | Variable rate | Size (MB) |
|---|---|---|---|---|---|---|
| IMG_0867.MOV | 90.1 | 2160×3840 | HEVC / AAC | 60.00 | No | 519.6 |
| IMG_4405.MOV | 18.6 | 3840×2160 | H.264 / AAC | 24.00 | No | 79.8 |

| File | Duration (s) | Resolution | Video / audio codec | Frame rate | Variable rate | Size (MB) |
|---|---|---|---|---|---|---|
| 43307CA1-…D043.mov | 7.5 | 1920×1080 | HEVC / AAC | 119.39 | Yes | 20.6 |
| Timeline One.mov | 12.5 | 3240×2160 | H.264 / AAC | 30.00 | No | 74.1 |
| VID_20260527_…314.MP4 | 6.3 | 1080×1920 | H.264 / AAC | 55.17 | Yes | 13.3 |
| 5977460-uhd_…25fps.mp4 | 13.0 | 3840×2160 | H.264 / none | 25.00 | No | 34.4 |

*Table 6. Sample records of the footage dataset*

Characteristics: duration and size vary by two orders of magnitude; portrait (2160×3840) and landscape clips are mixed; frame rates range from 24 to 119.39 fps, and two clips have a variable frame rate (average and base rate differ by more than 0.5 %), which the exporter must handle with rational rates. One stock clip has no audio track. Speech is sparse: 5 of 76 shots have three or more clearly transcribed words, because most clips are music, water or crowd noise or Kannada speech, which the small Whisper model cannot write reliably (it is marked 'detect only').

## 10. Proposed Approach

Fig. 1 CineMind AI system architecture and process flow

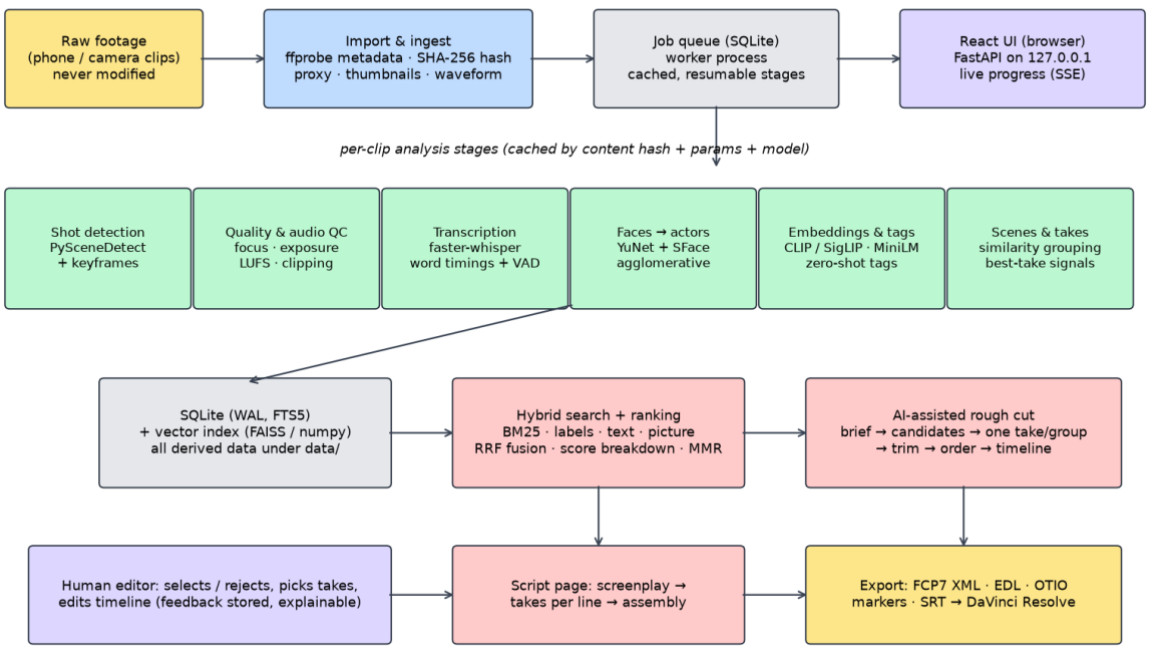



*Fig. 1. System architecture and process flow of CineMind AI*

- **Step 1.** Import: the editor chooses a folder inside an approved media root. Files are validated and hashed (SHA-256) and referenced in place; nothing is copied or modified.
- **Step 2.** Job queue and worker: one ingest job per clip runs cached stages, each keyed by (content hash, stage, parameter hash, model version), so re-running skips finished work and a crash resumes. Progress is pushed to the browser with server-sent events.
- **Step 3.** Per-clip analysis: proxy, thumbnails and waveform; shot detection and keyframes; quality and audio checks; transcription; faces; picture and text embeddings; tags.
- **Step 4.** Project-wide steps (after the last job): actor grouping, scene grouping, take grouping and vector-index rebuild, each skipped when its inputs are unchanged.

- **Step 5.** Storage: all derived data lives in SQLite (with FTS5) and under data/projects/<id>/, with provenance (model, version, thresholds) on every row.
- **Step 6.** Search and ranking: a query (text or picture) runs four retrievers, fused by RRF and re-ranked with an explained score breakdown.
- **Step 7.** Script and rough cut: a screenplay or a one-sentence brief turns ranked shots into a timeline (one take per group, speech-aware trims), which the editor edits in the browser.
- **Step 8.** Export: FCP7 XML, EDL, OTIO and SRT with reason markers, imported into DaVinci Resolve for finishing. Editor corrections at any step are stored and re-applied.

## 11. Result Analysis and Discussion

All figures below were produced by scripts in the project's eval/ folder on an Apple M1 (8 GB) CPU and are recorded with their settings in docs/EVALUATION.md. Where relevance or take labels were produced with an AI assistant (Claude) rather than by independent viewers, this is stated.

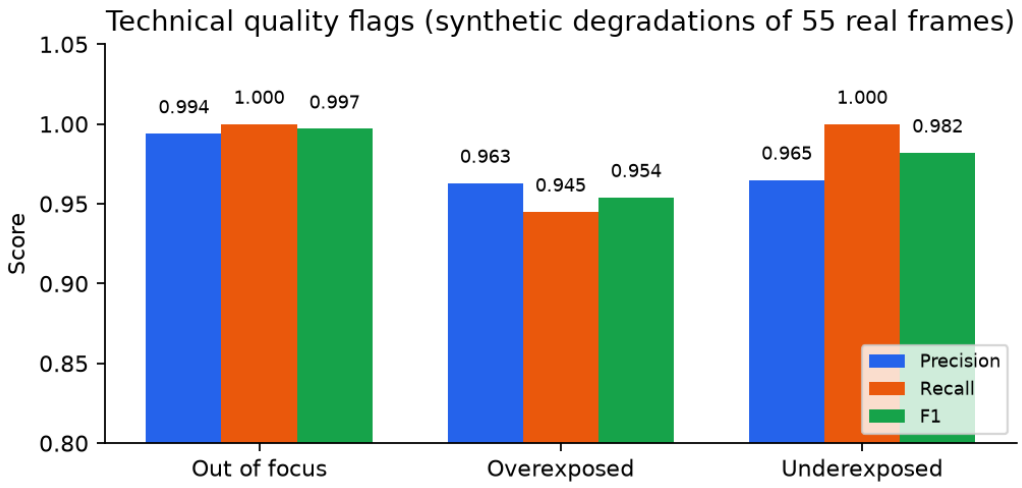



*Fig. 2. Precision, recall and F1 of the technical quality flags*

The focus flag (re-blur ratio with a Laplacian floor) reaches F1 0.997, better than plain Laplacian variance (0.985, Table 7). Exposure flags reach 0.954–0.982. The labels come from synthetic blur and exposure changes of 55 real frames, so these numbers are an upper bound. Fog, water and plain walls still read as soft in real footage.

| Flag / rule | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Out of focus (used: blur > 0.547 or sharpness < 5.32) | 0.994 | 1.000 | 0.997 | 0.995 |
| Out of focus, Laplacian variance alone (baseline) | 0.976 | 0.994 | 0.985 | 0.977 |
| Overexposed | 0.963 | 0.945 | 0.954 | 0.955 |
| Underexposed | 0.965 | 1.000 | 0.982 | 0.982 |

*Table 7. Quality flag calibration (5-fold grouped by clip)*

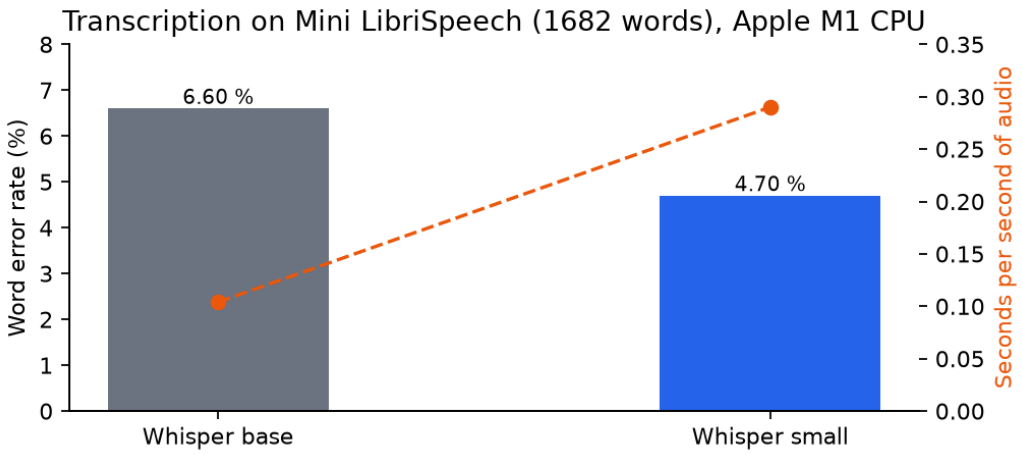



*Fig. 3. Word error rate and speed of the two Whisper sizes on Mini LibriSpeech*

Whisper small lowers WER from 6.60 % to 4.70 % at almost three times the processing time (0.29 s per second of audio, still about 3.4× faster than real time on CPU), so small is the default for the lite tier. On the team's noisy field audio, the uncertainty rule hid the hallucinated lines, but some fluent text that passes the model's own limits remains.

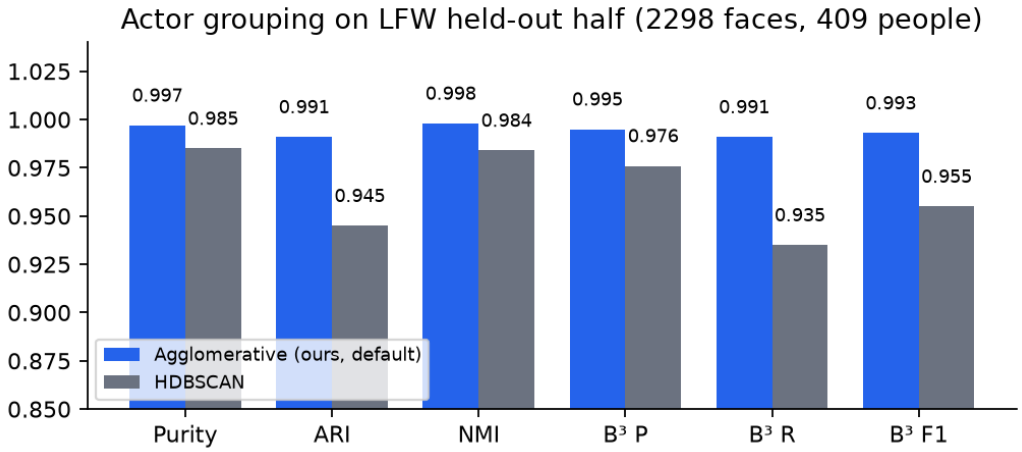



*Fig. 4. Actor grouping quality on LFW (held-out half)*

Average-linkage agglomerative clustering at distance 0.60 (chosen on the tuning half) gives 410 groups for 409 people, with B-cubed F1 0.993, ahead of HDBSCAN (0.955), which splits more people. LFW photos are frontal and well lit, so footage is harder. On our clips, setting aside faces seen in only one frame reduced 27 'actors' to 14.

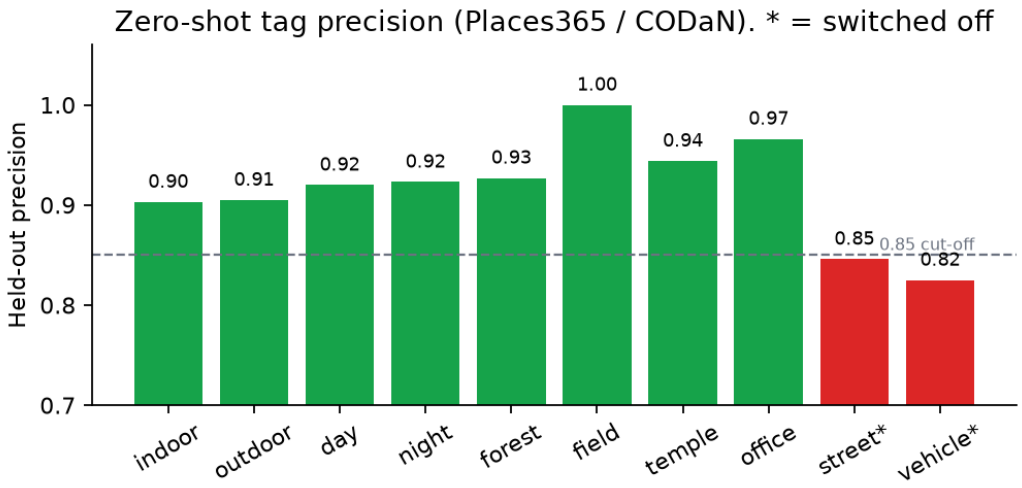



*Fig. 5. Held-out precision of the zero-shot tags*

Thresholds were set at 0.9 precision on the tuning half. Eight tags keep held-out precision of 0.90–1.00, while six tags such as 'street' (0.85) and 'inside a vehicle' (0.83) were switched off rather than shown wrongly. Recall is low for rare places (forest 0.41), so tags favour being right over being complete.

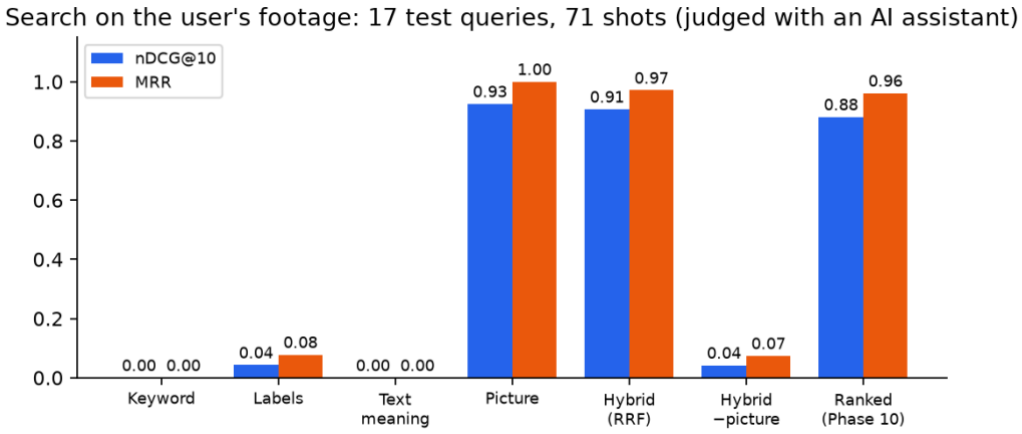



*Fig. 6. Search ablation on the team's footage (nDCG@10 and MRR)*

With 17 picture-oriented test queries, the picture list alone reaches nDCG@10 0.925 and MRR 1.0. The four-list fusion keeps almost all of that (0.907 / 0.971) while still answering spoken-word queries, and removing the picture list collapses quality (0.042). The explainable ranking with quality and duplicate penalty costs a little nDCG (0.881), because binary judgements cannot credit the extra variety. The relevance judgements were made with an AI assistant (Claude) from keyframes, not by independent viewers, so the numbers are indicative.

| Configuration | P@5 | P@10 | nDCG@10 | MRR |
|---|---|---|---|---|
| Keyword (BM25) | 0.000 | 0.000 | 0.000 | 0.000 |
| Labels | 0.047 | 0.024 | 0.043 | 0.078 |
| Meaning of speech | 0.000 | 0.000 | 0.000 | 0.000 |
| Picture | 0.588 | 0.376 | 0.925 | 1.000 |

| Configuration | P@5 | P@10 | nDCG@10 | MRR |
|---|---|---|---|---|
| Hybrid (RRF, all four) | 0.600 | 0.376 | 0.907 | 0.971 |
| Hybrid without picture | 0.047 | 0.024 | 0.042 | 0.074 |
| Ranked (weights, MMR) | 0.565 | — | 0.881 | 0.961 |

*Table 8. Search evaluation (test queries)*

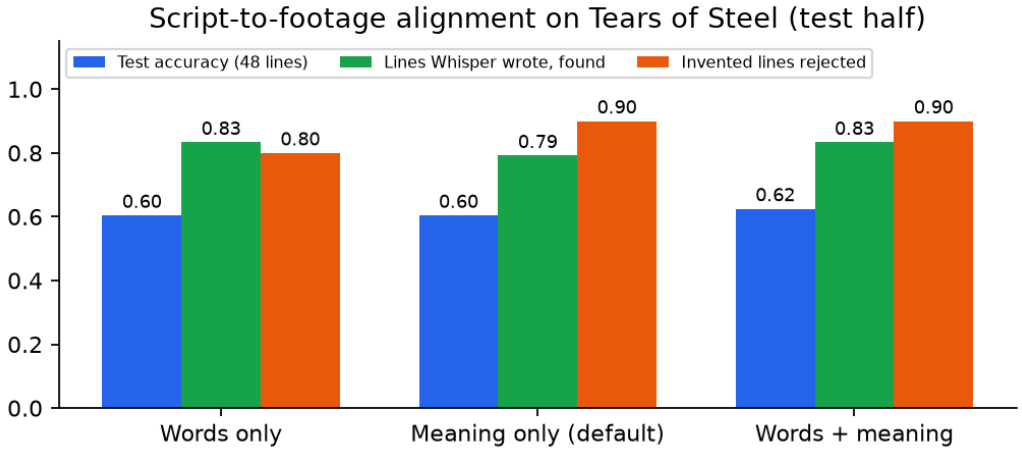



*Fig. 7. Script-to-footage alignment on Tears of Steel (test half)*

About 60 % of all script lines are placed correctly (29–30 of 48). Among lines that Whisper actually transcribed, 79–83 % are found, and 8–9 of 10 invented 'not filmed' lines are rejected. The missing lines (14 of 38 never transcribed) show that speech recognition, not the matcher, limits this feature. The 95 % intervals of the three variants overlap.

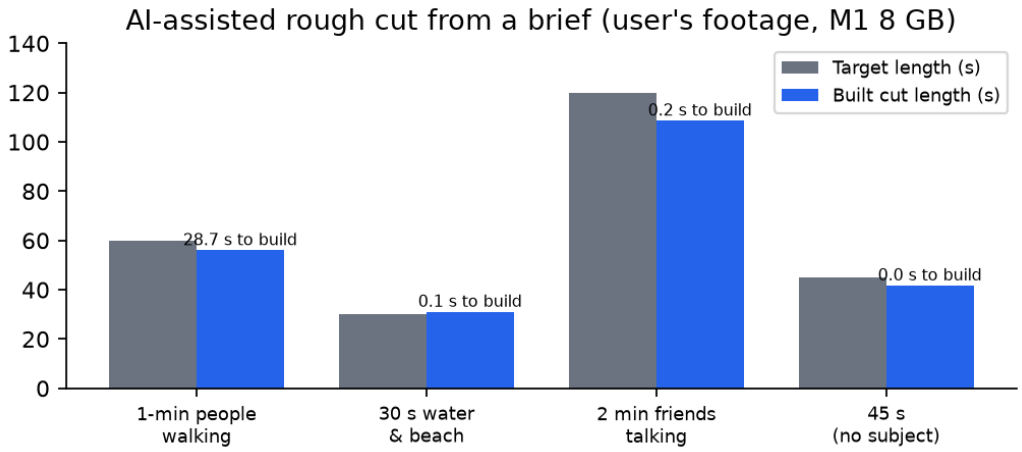



*Fig. 8. Rough cuts built from briefs on the team's footage*

Every cut lands within 10 % of the requested length (56.1 s for 60 s, 108.7 s for 120 s) because speech clips keep whole sentences. The first brief takes 28.7 s while the models load, and later briefs take 0.0–0.2 s. This measures speed and length only. Whether the chosen clips suit the brief is left to the editor.

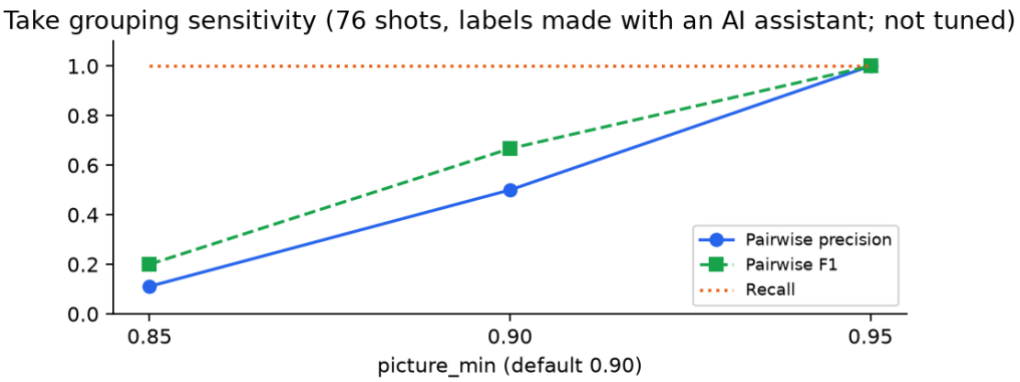



*Fig. 9. Take grouping on 76 shots: sensitivity to the picture limit*

With the default limits the system found the one labelled repeat (recall 1.0) and one false pair, a continuous shot split by a false cut (precision 0.5, B-cubed F1 0.993). Raising the picture limit to 0.95 removes the false pair, and lowering it to 0.85 groups unrelated neighbours (precision 0.11). The footage contains almost no retakes, so this is a sanity check and the defaults were not tuned on it.

Discussion. Three lessons stand out. First, for visual queries a calibrated picture embedding is the strongest single signal, and fusion protects against its blind spots (spoken words, names) at little cost. Second, speech recognition quality on field audio limits every dialogue feature (script alignment, speech-aware trimming, take grouping by words), so marking uncertain text is essential. Third, calibrating thresholds on labelled data and switching off unreliable outputs costs some recall but keeps the editor's trust. Exporting through FCP7 XML worked in DaVinci Resolve 21.1 (verified by hand), and the reason attached to each clip makes the AI-assisted rough cut reviewable.

## 12. Comparative Analysis

Table 9 compares CineMind with the surveyed systems on scope and method. Table 10 places our measured figures next to those reported in the literature. The datasets differ, so the numbers show the same order of magnitude rather than a head-to-head ranking.

| System | Runs locally on CPU | Search | Rough cut / assembly | Takes / script | Explains decisions | NLE export |
|---|---|---|---|---|---|---|
| CineMind AI (ours) | Yes (M1, 8 GB) | Text, speech, picture | Yes, from a brief | Yes, both | Yes, every score | FCP7 XML, EDL, OTIO |
| Premiere Text-Based Editing [16] | Inside Premiere | Transcript | Manual via text | No | No | Native only |
| Resolve 20 IntelliScript [17] | Resolve Studio (paid) | Limited | From a script | Script only | No | Native only |
| Jumper [18] | Yes | Text, speech, similar | No | No | Partly | Plug-in only |
| CLIP4Clip [5] / QVHighlights [14] | GPU | Text→video | No | No | No | No |
| Learning to Cut [12] / AVE [11] | GPU | No | Cut scoring / benchmark | No | No | No |

*Table 9. Functional comparison with surveyed tools and research*

| Component | Literature (reported, own dataset) | CineMind (measured through our pipeline) |
|---|---|---|
| Speech recognition | Whisper: robust zero-shot WER near supervised systems on LibriSpeech [1] | Whisper small via faster-whisper: 4.70 % WER on Mini LibriSpeech, 0.29 s per audio second on CPU |
| Word timestamps | WhisperX adds forced alignment, 12× faster batching [2] | Built-in VAD + word timings; uncertain lines hidden |
| Face detection / grouping | RetinaFace SOTA on WIDER FACE (GPU) [6]; YuNet millisecond-level [7] | YuNet + SFace on CPU: B-cubed F1 0.993 on LFW held-out |
| Zero-shot visual tags | CLIP / SigLIP zero-shot transfer, uncalibrated scores [3], [4] | Calibrated thresholds: 0.90–1.00 precision on 8 enabled tags |
| Text→video retrieval | CLIP4Clip state-of-the-art on MSR-VTT with fine-tuning [5] | No fine-tuning: nDCG@10 0.907, MRR 0.971 on our footage (judged with an AI assistant) |
| Scenes | LGSS multimodal model on MovieScenes [10] | Transparent similarity rule plus editor corrections (not evaluated against labels) |
| Script assembly | IntelliScript (closed, no published accuracy) [17] | 83 % of transcribed lines found; 90 % of invented lines rejected (Tears of Steel) |

*Table 10. Reported literature results versus CineMind measurements*

Inference: CineMind does not beat specialised GPU models on their own benchmarks and does not try to. What it adds is the combination: comparable component quality on a CPU laptop, calibrated and explained outputs, and a complete path from raw clips to an editable timeline in a free NLE. That combination is absent from both the research systems, which cover one task each, and the commercial tools, which are closed or locked to one editor.

# 13. User Manual

Start the application with `uv run cinemind serve` and open http://127.0.0.1:8000. The screenshots below were taken from the running application on the team's footage.

## 13.1 Library

Import a footage folder (referenced in place, never modified) and follow processing live. Filter and sort clips by technical quality.

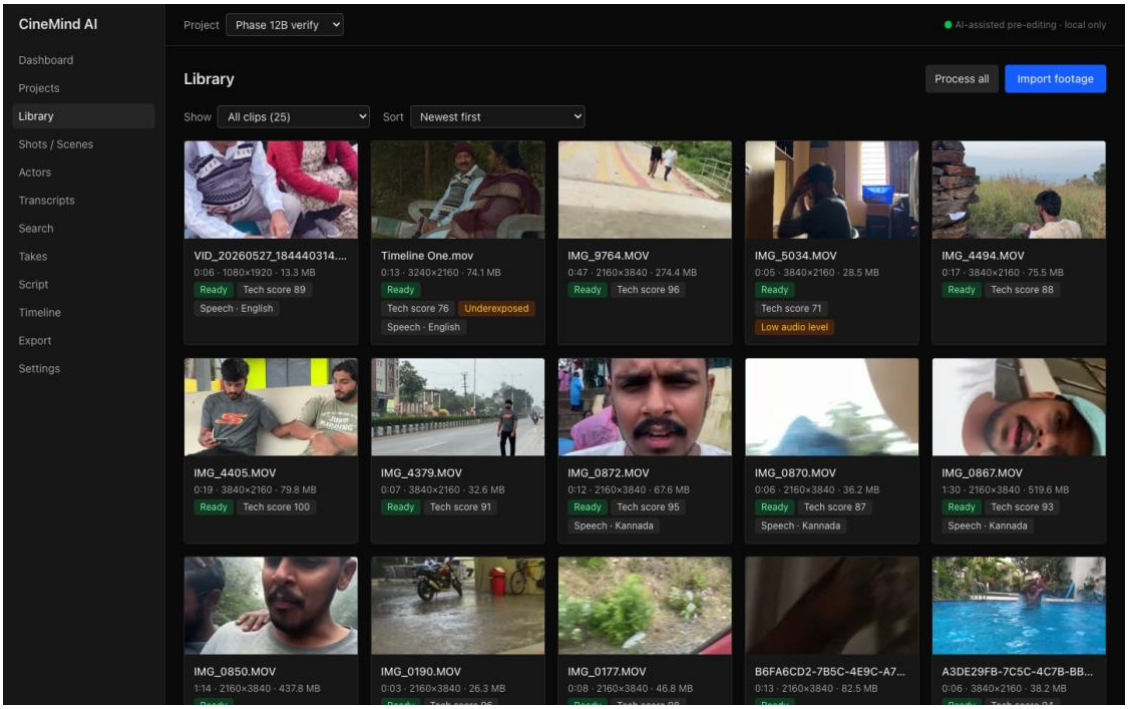



*Fig. 10. Library page*

## 13.2 Clip page

Proxy player with the shot strip, the QC lane of quality flags, the waveform, the transcript with word timings, tags and faces.

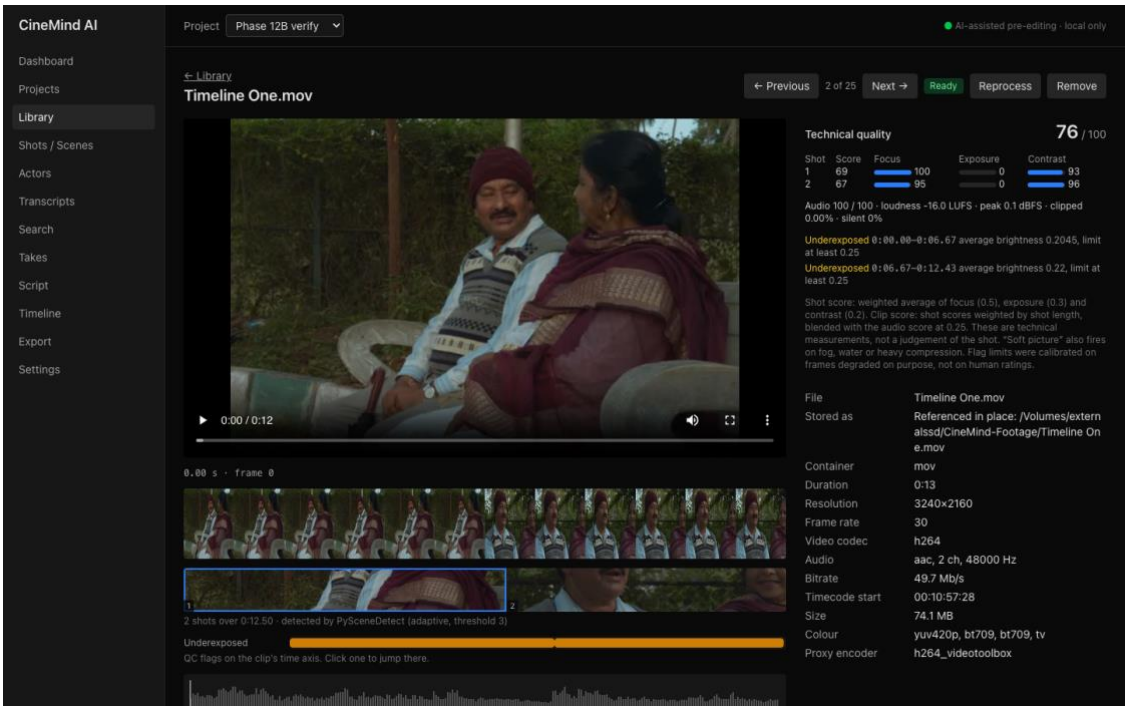



*Fig. 11. Clip page page*

## 13.3 Shots / Scenes

Shots grouped into scenes, with the similarity behind each boundary. Split, join and rename scenes, and select or reject shots.

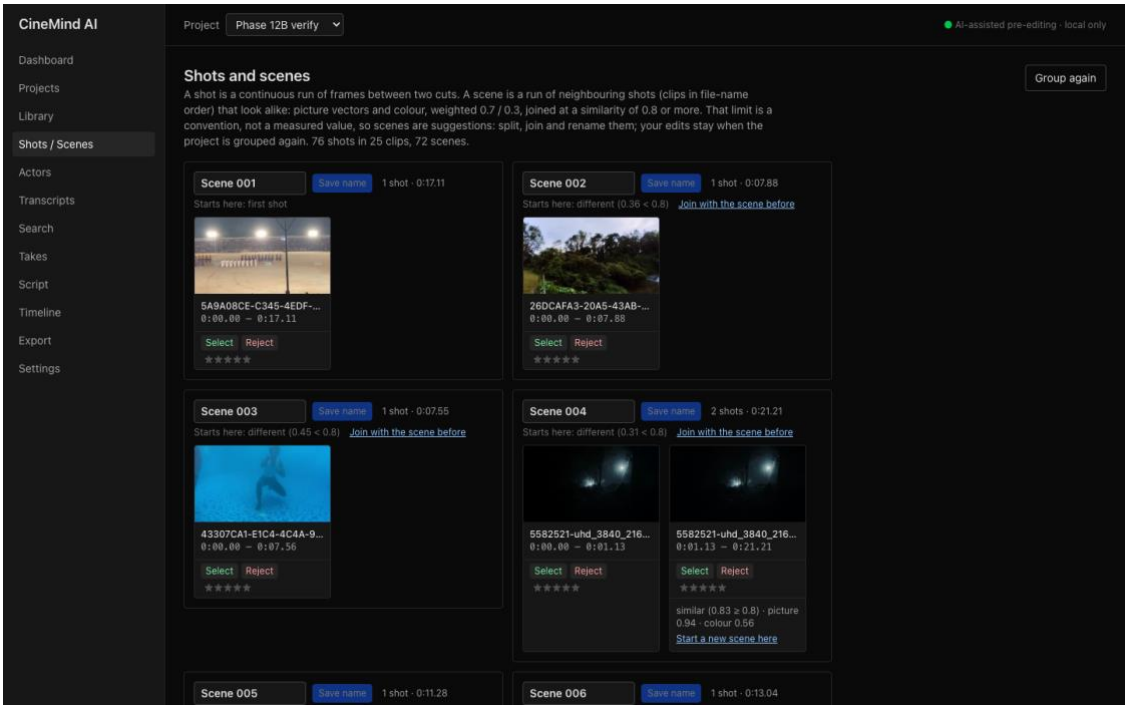



*Fig. 12. Shots / Scenes page*

## 13.4 Actors

Faces grouped into people. Rename, merge and split groups, and answer 'Same person?' questions.

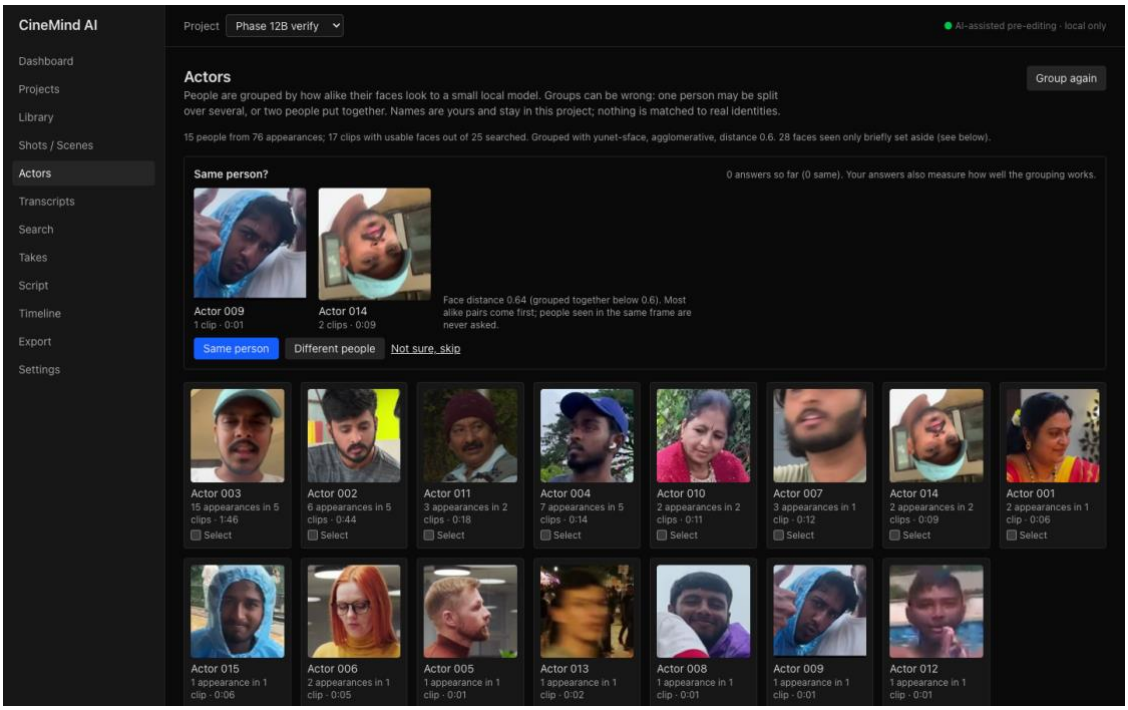



*Fig. 13. Actors page*

## 13.5 Transcripts

Spoken languages per project, the transcript of every clip and keyword search over all of them.

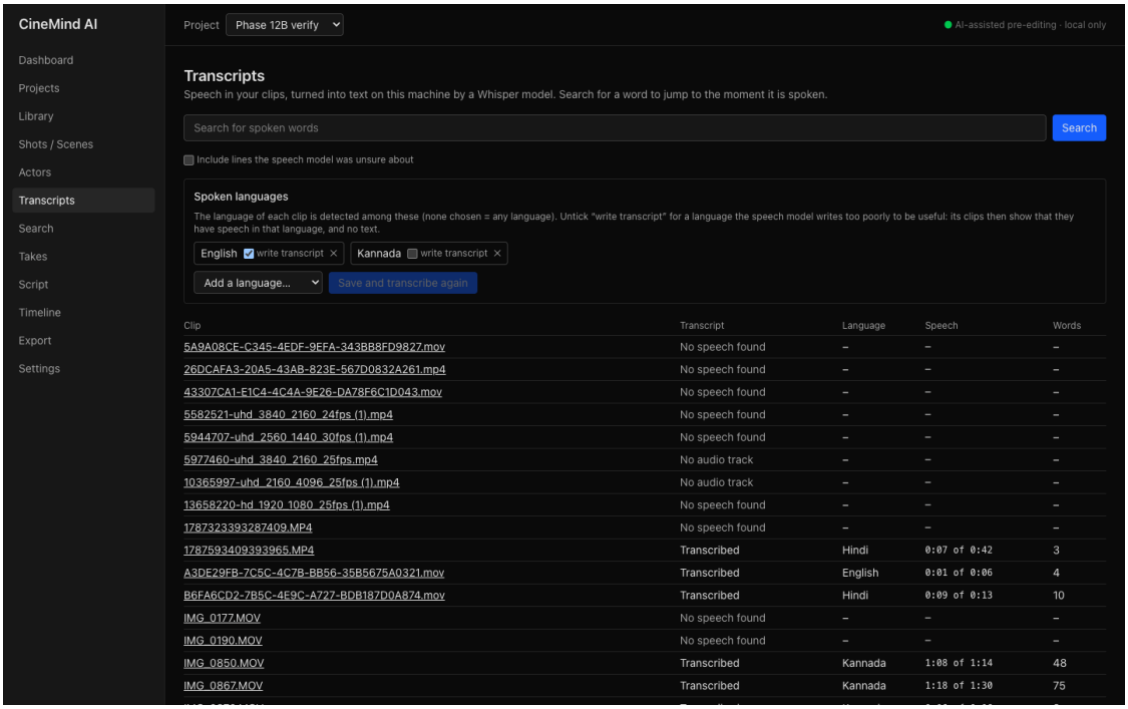



*Fig. 14. Transcripts page*

## 13.6 Search

Type a query or search by picture. Each card shows the score breakdown, and 'Ranking weights' changes the weights for one search.

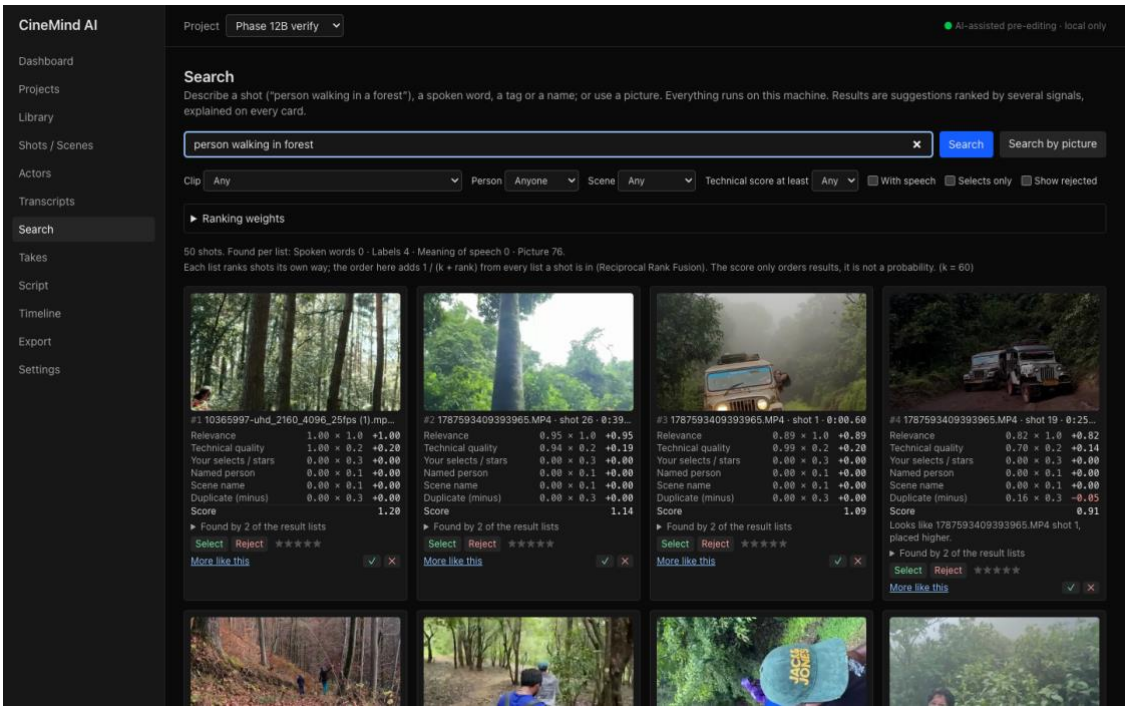



*Fig. 15. Search page*

## 13.7 Takes

Repeated takes with the suggested best take and its signals. Make another take the best, or take a shot out of the group.

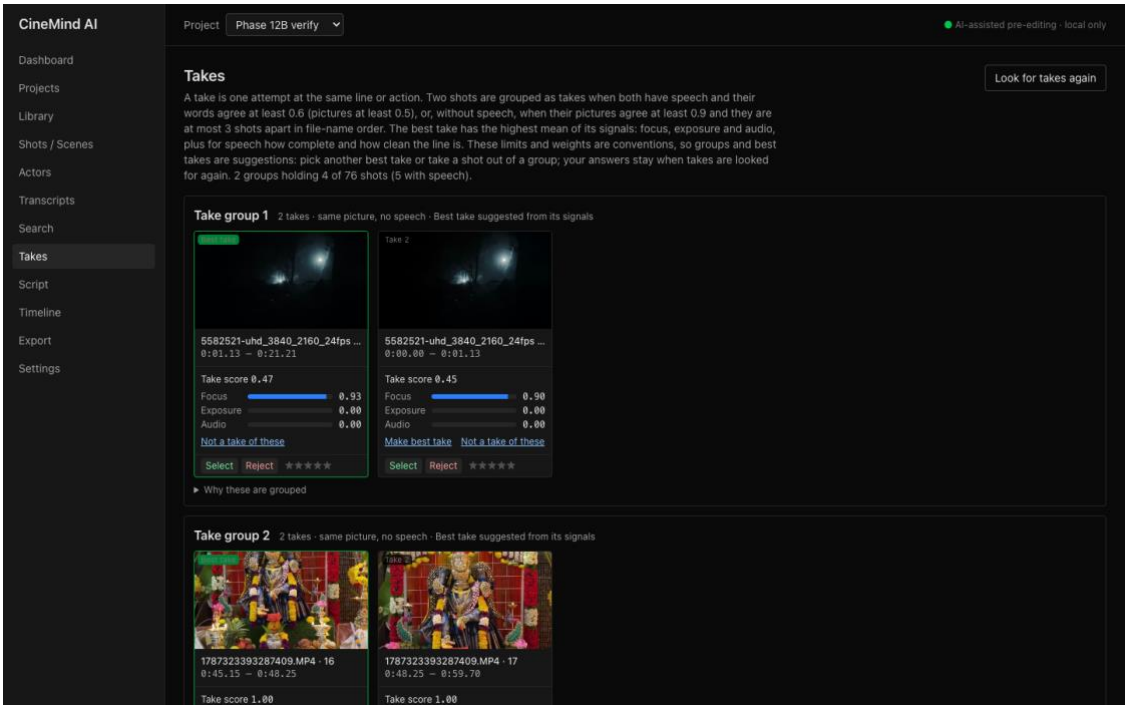



*Fig. 16. Takes page*

## 13.8 Script

Paste a screenplay. Every dialogue line gets its takes, and 'Build script assembly' creates a timeline.

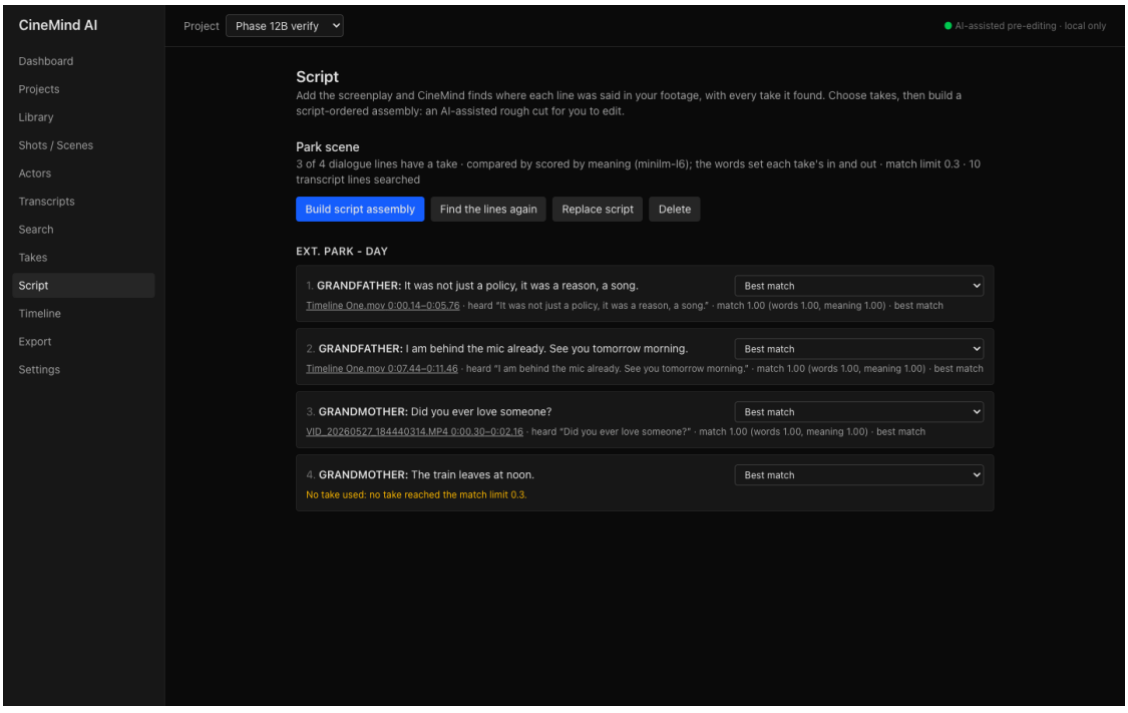



*Fig. 17. Script page*

## 13.9 Timeline

Write a brief and optionally set length, order, person and scene, then build an AI-assisted rough cut.

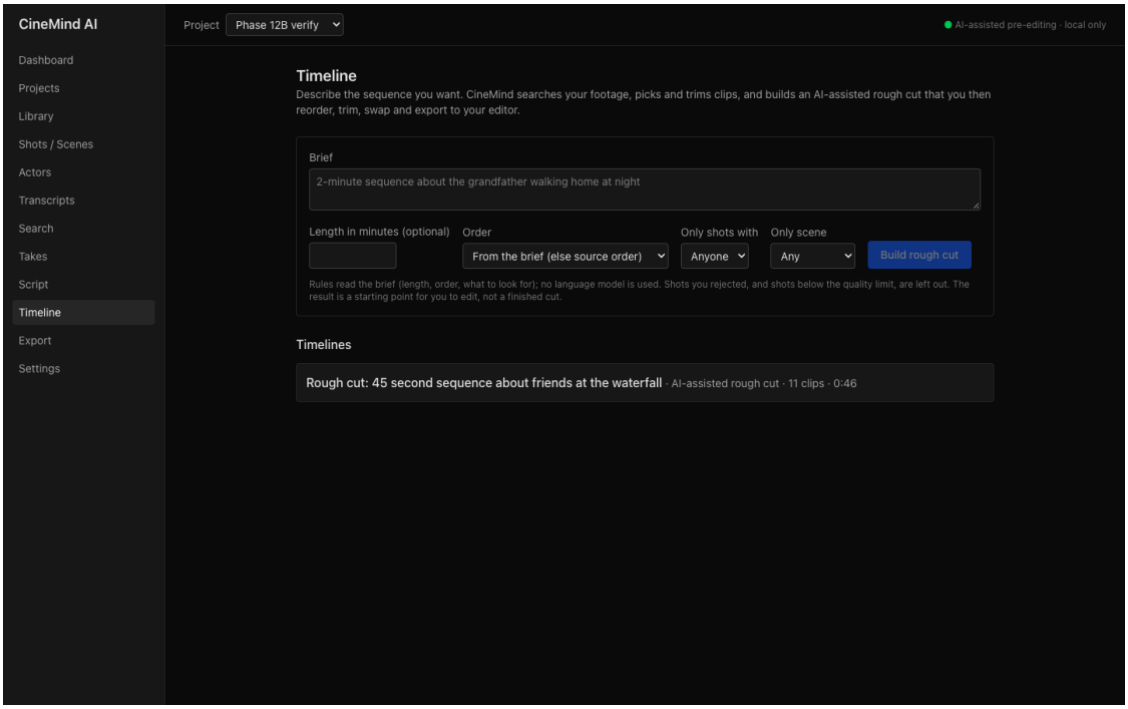



*Fig. 18. Timeline page*

## 13.10 Timeline editor

Reorder by drag-and-drop, trim, remove, swap with suggestions (other takes first) and play the cut. Every clip shows why it was chosen.

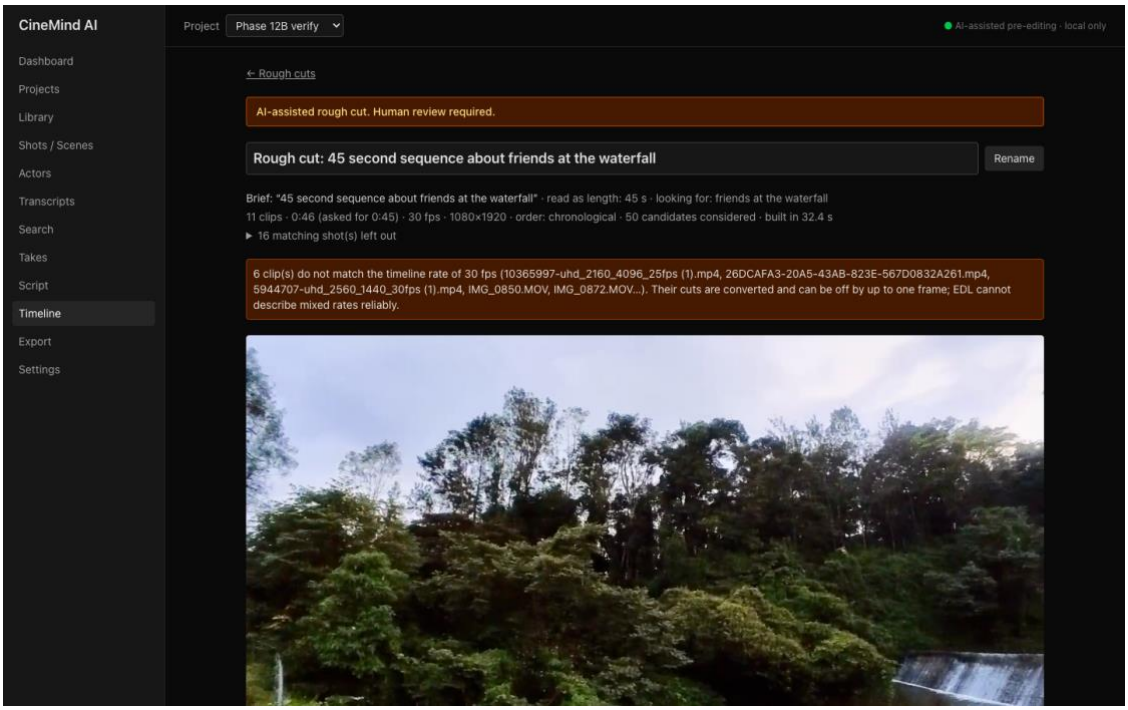



*Fig. 19. Timeline editor page*

## 13.11 Export

Export FCP7 XML (for DaVinci Resolve), EDL, OTIO and SRT. In Resolve choose File → Import → Timeline.

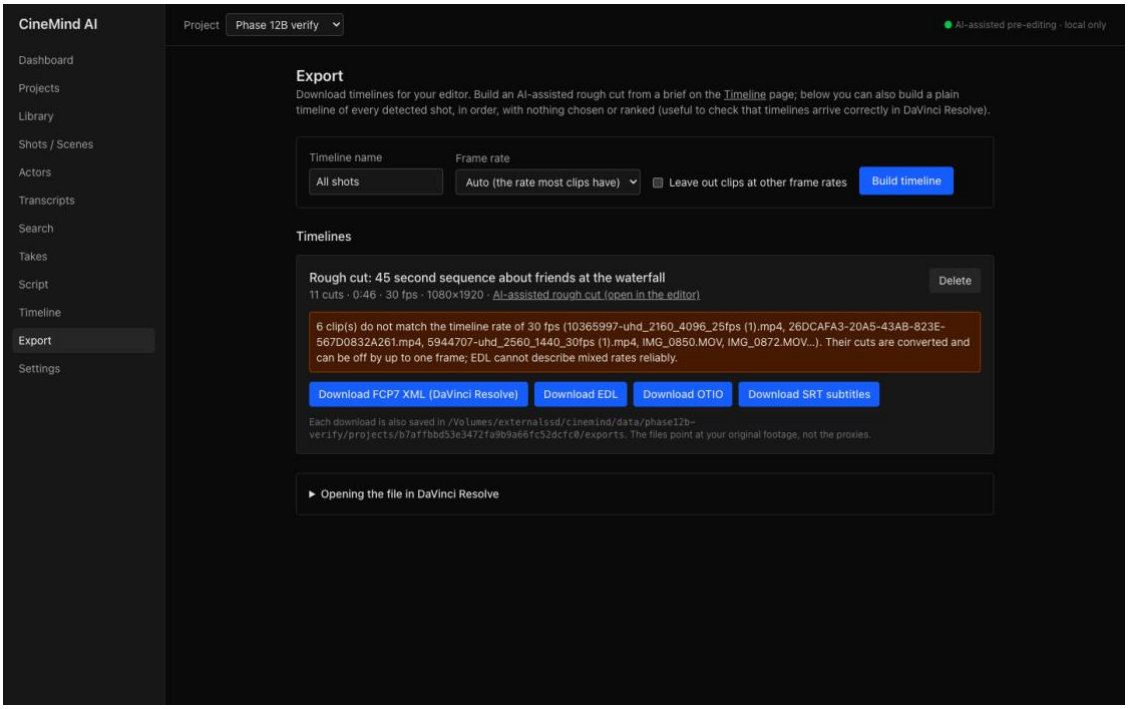



*Fig. 20. Export page*

## 14. References


[1] Radford, Alec, et al. "Robust Speech Recognition via Large-Scale Weak Supervision." Proceedings of the 40th International Conference on Machine Learning, PMLR, vol. 202, 2023, pp. 28492–28518.

[2] Bain, Max, et al. "WhisperX: Time-Accurate Speech Transcription of Long-Form Audio." Proceedings of Interspeech 2023, ISCA, 2023.

[3] Radford, Alec, et al. "Learning Transferable Visual Models from Natural Language Supervision." Proceedings of the 38th International Conference on Machine Learning, PMLR, vol. 139, 2021, pp. 8748–8763.

[4] Zhai, Xiaohua, et al. "Sigmoid Loss for Language Image Pre-Training." Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 11975–11986.

[5] Luo, Huaishao, et al. "CLIP4Clip: An Empirical Study of CLIP for End to End Video Clip Retrieval and Captioning." Neurocomputing, vol. 508, Elsevier, 2022.

[6] Deng, Jiankang, et al. "RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 5203–5212.

[7] Wu, Wei, Hanyang Peng, and Shiqi Yu. "YuNet: A Tiny Millisecond-Level Face Detector." Machine Intelligence Research, vol. 20, no. 5, Springer, 2023, pp. 656–665.

[8] Souček, Tomáš, and Jakub Lokoč. "TransNet V2: An Effective Deep Network Architecture for Fast Shot Transition Detection." arXiv preprint arXiv:2008.04838, 2020.

[9] Huang, Qingqiu, et al. "MovieNet: A Holistic Dataset for Movie Understanding." Computer Vision – ECCV 2020, Springer, 2020.

[10] Rao, Anyi, et al. "A Local-to-Global Approach to Multi-Modal Movie Scene Segmentation." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 10146–10155.

[11] Argaw, Dawit Mureja, et al. "The Anatomy of Video Editing: A Dataset and Benchmark Suite for AI-Assisted Video Editing." Computer Vision – ECCV 2022, Springer, 2022, pp. 201–218.

[12] Pardo, Alejandro, et al. "Learning to Cut by Watching Movies." Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 6858–6868.

[13] Shen, Yaojie, et al. "AutoTransition: Learning to Recommend Video Transition Effects." Computer Vision – ECCV 2022, Springer, 2022, pp. 285–300.

[14] Lei, Jie, Tamara L. Berg, and Mohit Bansal. "Detecting Moments and Highlights in Videos via Natural Language Queries." Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021.

[15] Wang, Wenhui, et al. "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers." Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[16] Adobe. "NAB 2023: Introducing Text-Based Editing in Premiere Pro, Properties Panel in After Effects, and Much More." Adobe Blog, 13 Apr. 2023, blog.adobe.com/en/publish/2023/04/13/nab-2023-introducing-text-based-editing-premiere-pro-properties-panel-after-effects-more.

[17] "Blackmagic Introduces DaVinci Resolve 20." ProVideo Coalition, 2025, www.provideocoalition.com/blackmagic-introduces-davinci-resolve-20/.

[18] "Review: AI-Driven Video Search with Jumper." ProVideo Coalition, 2024, www.provideocoalition.com/review-ai-driven-video-search-with-jumper/.

[19] Castellano, Brandon. PySceneDetect: Video Scene Cut Detection and Analysis Tool. 2024, www.scenedetect.com.

[20] Academy Software Foundation. OpenTimelineIO: Open Source API and Interchange Format for Editorial Timeline Information. 2024, opentimeline.io.

[21] Cormack, Gordon V., Charles L. A. Clarke, and Stefan Büttcher. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods." Proceedings of the 32nd International ACM SIGIR Conference, 2009, pp. 758–759.

[22] SYSTRAN. faster-whisper: Faster Whisper Transcription with CTranslate2. GitHub, 2024, github.com/SYSTRAN/faster-whisper.

## 15. Appendix

- **a) Presentation (PPT):** [paste the PPT link here]
- **b) Pre-recorded demonstration video:** [paste the demonstration video link here]
- **c) Source code and steps to execute:** https://github.com/rishith-gg/cinemind-ai (README.md, section 'Run it'; full guide in docs/USER_GUIDE.md)

### Steps to execute

1. `git clone <repository> cinemind && cd cinemind`
2. `uv sync  and  (cd frontend && pnpm install)`
3. `uv run cinemind doctor  (checks ffmpeg, Python, Node, RAM, GPU)`
4. `uv run cinemind models pull  (downloads models once)`
5. `Set CINEMIND_MEDIA_ROOTS in .env to the footage folder`
6. `uv run python scripts/tasks.py build-ui`
7. `uv run cinemind serve  → open http://127.0.0.1:8000`